1. El lenguaje pLTL

La gramática de LTL implementada hasta el momento en MTSA es:

$$
\begin{array}{|lll|l}
\pi^i \models \top & & & \\
\pi^i \models \neg\varphi & \iff & \pi^i \not\models \psi & \\
\pi^i \models \varphi \wedge \psi & \iff & \pi^i \models \varphi \text{ y } \pi^i \models \psi & \\
\pi^i \models \varphi \vee \psi & \iff & \pi^i \models \varphi \text{ o } \pi^i \models \psi & \\
\pi^i \models \bigcirc\varphi & \iff & \pi^{i+1} \models \varphi & \textit{(Next, Siguiente)} \\
\pi^i \models \varphi\, \mathcal{U}\, \psi & \iff & \exists k \geq i / \pi^k \models \psi \text{ y } \forall i \leq j < k, \pi^j \models \varphi & \textit{(Until, Hasta)} \\
\pi^i \models \varphi\, \mathcal{R}\, \psi & \iff & \forall k \geq i, \pi^k \models \psi \text{ o } \exists i \leq j < k / \pi^j \models \varphi & \textit{(Release, Libera)}
\end{array}
$$

De estos símbolos se desprenden las definiciones:

$$
\begin{array}{|lll}
\bot & \equiv \neg\top & \textit{(Falso)} \\
\Diamond\varphi & \equiv \top\, \mathcal{U}\, \varphi & \textit{(Eventualmente } \varphi\textit{)} \\
\Box\varphi & \equiv \neg\Diamond\neg\varphi & \textit{(Siempre } \varphi\textit{)} \\
\varphi\, \mathcal{W}\, \psi & \equiv \psi\, \mathcal{R}\, (\varphi \vee \psi) & \textit{(Until débil)}
\end{array}
$$

Estos últimos símbolos son parte del lenguaje LTL al que tiene acceso el usuario, pero el parser los transforma inmediatamente a su equivalente con los operadores de la primera tabla.

[] utiliza los operadores $\top, \neg, \wedge, \bigcirc, \mathcal{U}$, y agrega los operadores de pasado:

$$
\begin{array}{|lll|l}
\pi^i \models \bullet\varphi & \iff & i > 0 \text{ y } \pi^{i-1} \models \varphi & \textit{(Yesterday, Anterior)} \\
\pi^i \models \varphi\, \mathcal{S}\, \psi & \iff & \exists k, 0 \leq k \leq i / \pi^k \models \psi \text{ y } \forall k \leq j < i, \pi^j \models \varphi & \textit{(Since, Desde)}
\end{array}
$$

De los cuales, de la misma forma que en futuro, por comodidad del usuario, agregamos las definiciones:

$$
\begin{array}{|lll}
\blacklozenge\varphi & \equiv \top\, \mathcal{S}\, \varphi & \textit{(Eventualmente, en el pasado, } \varphi\textit{)} \\
\blacksquare\varphi & \equiv \neg\blacklozenge\neg\varphi & \textit{(Siempre, en el pasado, } \varphi\textit{)}
\end{array}
$$

Por simplicidad, MTSA mantiene siempre las fórmulas en forma normal negada, es decir que el $\neg$ solo puede estar antes de una proposición. Para poder mantener esta restricción, cuando se crea una fórmula con una negación, se aplica la transformación recursiva:

$\neg X\varphi$	$\rightarrow$	$\tilde{X}\neg\varphi$	Si X es un operador unario y $\tilde{X}$ su dual
$\neg(\psi X\varphi)$	$\rightarrow$	$\neg\psi\tilde{X}\neg\varphi$	Si X es un operador binario y $\tilde{X}$ su dual
$\neg\varphi$	$\rightarrow$	$\neg\varphi$	Si φ es una proposición

Queda claro entonces que es necesario que todo operador tenga un dual. Para todos los operadores de futuro ya presentes, existe en el lenguaje un dual ($\mathcal{U}$ y $\mathcal{R}$ son duales, y $\bigcirc$ es su propio dual). Pero [] no presenta operadores duales para los símbolos de pasado que introduce.

Para el $\mathcal{S}$, agregamos al lenguaje su dual $\mathcal{T}$, definido como:

$$
\begin{array}{|lll|l}
\pi^i \models \varphi\, \mathcal{T}\, \psi & \iff & \forall 0 \leq k \leq i, \pi^k \models \psi \text{ o } \exists k < j \leq i / \pi^j \models \varphi & \textit{(Trigger, Gatilla)}
\end{array}
$$

A diferencia de su equivalente de futuro, no es verdad que $\bullet$ es su propio dual. Esto se debe a la condición agregada $i > 0$ que hace que $\pi^0 \not\models \bullet\varphi$ para cualuier π y φ. Entonces no es cierto que $\neg\bullet\varphi \equiv \bullet\neg\varphi$, porque en el momento 0, $\pi^0 \models \neg \bullet \varphi$ pero $\pi^0 \not\models \bullet\neg\varphi$. Por eso es necesario agregar el operador $\tilde{\bullet}$, con semántica:

$$
\begin{array}{|lll|l}
\pi^i \models \tilde{\bullet}\varphi & \iff & i = 0 \text{ o } \pi^{i-1} \models \varphi & \textit{(Yesterday débil)}
\end{array}
$$

Es interesante notar que, evaluados en cualquier momento que no es el inicial, $\bullet$ y $\tilde{\bullet}$ son equivalentes.

Agregando $\mathcal{T}$ y $\tilde{\bullet}$ a los operadores usados en [] y MTSA, obtenemos un lenguaje de lógica temporal con operadores de pasado y futuro que podemos mantener en forma normal negada.